

Entornos de Desarrollo · RA3

MEMORIA DEL PROYECTO SMARTFENIX

Verificación y pruebas de una aplicación Spring Boot

Actividad

Entornos de Desarrollo

Resultado de aprendizaje

RA3

Alumno

Sergio Hidalgo López

Curso

1º DAM

Proyecto

SmartFenix

Fecha

Junio 2026

Índice

Resumen visual de los apartados principales incluidos en la memoria final.

1. Resumen del proyecto

2. Tecnologías utilizadas

3. Arquitectura del sistema

4. Entidades principales

5. Configuración del entorno

6. Interfaz web y API REST

7. Casos de prueba

8. Pruebas unitarias

9. Prueba de integración

10. Depuración del sistema

11. Resultados

12. Incidencias y conclusión

Resumen visual del proyecto

Vista rápida de los datos más importantes del proyecto y del estado final de las pruebas.

Proyecto

SmartFenix

Tecnología principal

**Spring Boot
3.2.5 + Java
21**

Base de datos real

**MySQL 8.0 en
Docker**

Resultado de
pruebas

**26 tests
superados**

Configuración principal

Aplicación: http://localhost:8099

MySQL: localhost:3308

phpMyAdmin: http://localhost:8090

Credenciales: root / root

Resultado de pruebas

BUILD SUCCESS

Tests run: 26

Failures: 0

Errors: 0

Skipped: 0

Descripción general

Explicación breve del objetivo del proyecto, del problema que resuelve y de sus dos formas principales de uso.

SmartFenix es una aplicación desarrollada con Spring Boot para gestionar clientes, empleados, proyectos y tareas de una forma más organizada. La idea principal del proyecto es reunir en una sola aplicación información que normalmente estaría repartida en distintos sitios, de manera que sea más fácil consultarla, modificarla y comprobar su estado.

El proyecto resuelve el problema de llevar el control de datos empresariales de forma desordenada. En lugar de depender de herramientas separadas, SmartFenix permite centralizar clientes o empresas, empleados internos, proyectos asociados a un cliente y tareas asignadas a proyectos y empleados.

Desde la aplicación, el usuario puede crear registros nuevos, consultar información guardada, modificar datos existentes y eliminar registros cuando sea necesario. La interfaz web con Thymeleaf está pensada para trabajar desde el navegador, mientras que la API REST está pensada para realizar peticiones HTTP desde herramientas como Postman o desde pruebas automáticas.

Tecnologías utilizadas

Tabla resumen de las tecnologías que forman el proyecto y del papel que cumple cada una dentro de la aplicación y de las pruebas.

Tecnología	Versión	Explicación
Java	21	Lenguaje principal del proyecto.
Spring Boot	3.2.5	Permite crear la aplicación de forma rápida y organizada.
Maven	-	Gestiona dependencias, compilación y pruebas.
Spring Data JPA	-	Facilita el acceso a datos mediante repositorios.
Spring Validation	-	Ayuda a validar formularios y peticiones.
Lombok	1.18.32	Reduce código repetitivo.
MySQL	8.0	Base de datos real usada por la aplicación.
Docker Compose	-	Levanta MySQL y phpMyAdmin.
phpMyAdmin	-	Permite visualizar la base de datos.
Thymeleaf	-	Se usa para la interfaz web.
JUnit 5	-	Framework principal de pruebas.
Mockito	-	Simula repositorios en unitarias.
MockMvc	-	Simula peticiones HTTP en integración.
H2	-	Base de datos en memoria para integración.

Arquitectura del sistema

La aplicación sigue una arquitectura por capas para separar responsabilidades y facilitar mantenimiento, pruebas y comprensión del flujo interno.

Capas principales

Domain: contiene las entidades JPA y representa la estructura principal de los datos.

Repository: contiene interfaces que extienden `JpaRepository` y permiten guardar, buscar, listar y eliminar registros.

Service: concentra la lógica de negocio y evita que el controlador acceda directamente al repositorio.

Controller: recibe peticiones HTTP y devuelve vistas web o respuestas JSON.

Arquitectura general

Usuario / Navegador /
Postman



Controller



Service



Repository



Base de datos MySQL

Pruebas unitarias

Service



Repository simulado con Mockito

Entidades principales

Las cuatro entidades centrales del sistema representan la información principal que se guarda y se gestiona en la aplicación.

Entidad	Campos principales	Función
Cliente	nombre, empresa, telefono	Representa una empresa o persona que solicita trabajos.
Empleado	nombre, email, rol	Representa trabajadores internos.
Proyecto	nombre, fechaInicio, fechaFin, cliente	Representa trabajos o proyectos asociados a un cliente.
Tarea	descripcion, estado, proyecto, empleado	Representa trabajos concretos dentro de un proyecto.

Configuración del entorno

Resumen de la configuración necesaria para ejecutar la aplicación real y de la separación entre el entorno de uso normal y el entorno de pruebas.

Configuración técnica

- **Java 21** para compilar y ejecutar la aplicación.
- **Maven** para dependencias y comandos como `mvn test` o `mvn spring-boot:run`.
- **Docker Compose** para levantar MySQL y phpMyAdmin.
- **MySQL real** para la ejecución normal de la aplicación.
- **H2** solo para la prueba de integración.
- **phpMyAdmin** para visualizar la base de datos desde el navegador.

Direcciones principales

Aplicación: `http://localhost:8099`

MySQL: `localhost:3308`

phpMyAdmin: `http://localhost:8090`

Usuario: `root`

Contraseña: `root`

Comandos principales

```
docker compose up -d
docker compose ps
mvn test
mvn spring-boot:run
```

Interfaz web y API REST

La aplicación puede usarse desde el navegador o mediante peticiones HTTP, lo que facilita tanto la demostración visual como la verificación técnica.

Interfaz web

La interfaz web permite gestionar clientes, empleados, proyectos y tareas directamente desde el navegador.

- /
- /dashboard
- /clientes
- /empleados
- /proyectos
- /tareas

Cada sección permite listar, crear, editar y eliminar registros.

API REST

La API REST permite trabajar con Postman o con pruebas automáticas como MockMvc.

- **GET** consulta datos
- **POST** crea datos
- **PUT** actualiza datos
- **DELETE** elimina datos

Todos los endpoints principales están disponibles bajo /api.

Casos de prueba

Se han definido cinco casos de prueba que cubren operaciones funcionales básicas y un caso de error controlado.

Caso	Qué se prueba	Datos de entrada	Resultado esperado
Alta de cliente	Crear un cliente	Nombre, empresa y teléfono	El cliente se guarda y aparece en el listado.
Consulta de clientes	Consultar clientes existentes	GET /api/clientes o ruta /clientes	Se muestra la lista de clientes.
Modificación de proyecto	Editar un proyecto	Nuevo nombre o fechas	El proyecto queda actualizado.
Eliminación de empleado	Eliminar un empleado existente	ID de empleado	El empleado desaparece del sistema.
Consulta de registro inexistente	Consultar un ID inexistente	GET /api/empleados/9999	Respuesta 404 Not Found.

Pruebas unitarias

Las pruebas unitarias revisan la lógica de negocio de forma aislada, sin arrancar toda la aplicación y sin depender de una base de datos real.

Configuración usada

- JUnit 5
- Mockito
- `@ExtendWith(MockitoExtension.class)`
- `@Mock`
- `@InjectMocks`

Estas pruebas no usan base de datos real, no usan MySQL, no usan Docker y no usan `@SpringBootTest`.

Testing unitario

Service



Repository simulado
con Mockito

Clase de test	Métodos comprobados	Qué verifica
ClienteServiceTest	testCrearCliente, testListarClientes, testEliminarCliente, testNoEliminarClienteConProyectosAsociados	Guardado, listado, borrado correcto y bloqueo por proyectos asociados.
EmpleadoServiceTest	testCrearEmpleado, testListarEmpleados, testEliminarEmpleado, testNoEliminarEmpleadoConTareasAsociadas	Guardado, listado, borrado correcto y bloqueo por tareas asociadas.
ProyectoServiceTest	testCrearProyecto, testListarProyectos, testActualizarProyecto, testEliminarProyecto, testNoEliminarProyectoConTareasAsociadas	Creación, consulta, actualización y control de dependencias al borrar.

Clase de test	Métodos comprobados	Qué verifica
TareaServiceTest	testCrearTarea, testListarTareas, testEliminarTarea, testEliminarTareaNoExistente, testActualizarTarea	Alta, consulta, borrado, error controlado y actualización de tareas.

Prueba de integración

Las pruebas de integración comprueban que varias capas funcionan juntas dentro del contexto de Spring, usando H2 en memoria y MockMvc.

Configuración

- Spring Boot Test
- MockMvc
- @SpringBootTest
- @AutoConfigureMockMvc
- @ActiveProfiles("test")
- H2 en memoria

Testing integración

MockMvc



Controller



Service



Repository



H2 en memoria

Prueba	Qué comprueba
testListarEmpleadosDevuelveOk	Que GET /api/empleados responde correctamente.
testCrearEmpleadoFlujoCompleto	Que se puede crear un empleado mediante POST.
testConsultarEmpleadoInexistenteDevuelveNotFound	Que un ID inexistente devuelve 404.
testEliminarEmpleadoConTareasDevuelveConflict	Que un empleado con tareas asociadas devuelve 409 Conflict.
testListarClientesDevuelveOk	Que GET /api/clientes responde correctamente.

Prueba	Qué comprueba
<code>testCrearClienteDevuelveCreated</code>	Que se puede crear un cliente mediante POST.
<code>testConsultarClienteInexistenteDevuelveNotFound</code>	Que un ID inexistente devuelve 404.
<code>testEliminarClienteConProyectosDevuelveConflict</code>	Que un cliente con proyectos asociados devuelve 409 Conflict.

Depuración del sistema

La depuración ayuda a entender el flujo interno del programa y a revisar cómo cambian los valores durante la ejecución. El caso principal recomendado es la eliminación de un cliente con proyectos asociados.

1. Poner breakpoints en `ClienteWebController.eliminar(...)` y `ClienteService.delete(...)`.
2. Arrancar la aplicación y entrar en `http://localhost:8099/clientes`.
3. Intentar eliminar un cliente con proyectos asociados.
4. Avanzar paso a paso con `Step Over`.
5. Inspeccionar `id`, la comprobación de existencia, la comprobación de proyectos asociados y la excepción controlada.
6. Comprobar que la web muestra un mensaje controlado y no aparece error 500.

Ejecución de pruebas y resultados

Las pruebas pueden ejecutarse tanto desde terminal como desde IntelliJ IDEA. El resultado final confirma que la suite completa pasa correctamente y que la aplicación responde bien en sus rutas principales.

Ejecución

```
mvn test
```

También se verificó el arranque real con `mvn spring-boot:run` y la comprobación HTTP de web y API.

Resultado

BUILD SUCCESS

Tests run: 26

Failures: 0

Errors: 0

Skipped: 0

Rutas comprobadas

`/`, `/dashboard`, `/clientes`, `/empleados`, `/proyectos` y `/tarefas` respondieron correctamente.

También respondieron correctamente `/api/clientes`, `/api/empleados`, `/api/proyectos` y `/api/tareas`.

Resultado funcional

La aplicación arranca en `http://localhost:8099`.

No se produjeron errores 500 en los flujos de eliminación controlada tras la corrección final.

El sistema queda listo para demostración y entrega.

Incidencias encontradas y soluciones

Durante la revisión de la documentación y de la ejecución del proyecto aparecieron varias incidencias reales que se resolvieron antes de la entrega.

Incidencia	Causa	Solución
Documentación antigua contradictoria	Existían referencias antiguas en la memoria y en otros textos del proyecto.	Revisar la documentación y dejar una única versión coherente.
Confusión entre MySQL real y H2	Podía mezclarse la base de datos de la aplicación con la de pruebas.	Aclarar que MySQL es para la app real y H2 para integración.
Eliminación directa de registros relacionados	Algunas entidades no podían borrarse si tenían relaciones activas por claves foráneas.	Comprobar dependencias antes de eliminar y devolver mensajes controlados.
Error 500 en /api/proyectos y /api/tareas	Jackson intentaba serializar proxies lazy de Hibernate.	Ignorar metadatos internos de Hibernate en las entidades para devolver JSON válido.

Eliminación controlada de registros relacionados

Durante las pruebas se detectó que algunos registros no podían eliminarse directamente porque estaban relacionados con otras entidades. Esto ocurre por las claves foráneas de la base de datos, que protegen la integridad de la información.

Casos detectados

- Un cliente no puede eliminarse si tiene proyectos asociados.
- Un empleado no puede eliminarse si tiene tareas asignadas.
- Un proyecto no puede eliminarse si tiene tareas asociadas.

Solución aplicada

- Comprobar si existen registros relacionados antes de eliminar.
- Bloquear la eliminación si existen dependencias.
- Mostrar mensajes claros en web y respuestas controladas en la API.

Registro	Dependencia revisada	Resultado si tiene dependencia	Resultado si no tiene dependencia
Cliente	Proyectos asociados	No se elimina y se muestra mensaje.	Se elimina correctamente.
Empleado	Tareas asignadas	No se elimina y se muestra mensaje.	Se elimina correctamente.
Proyecto	Tareas asociadas	No se elimina y se muestra mensaje.	Se elimina correctamente.
Tarea	Sin dependencias principales	Se elimina normalmente.	Se elimina correctamente.

Verificación de DELETE en web y API

Se comprobó el comportamiento real tanto desde la interfaz web como desde la API REST para asegurar que los errores quedan controlados y que los borrados válidos se completan correctamente.

Interfaz web

Desde los listados se verificó que el usuario puede intentar eliminar registros y recibir siempre una respuesta controlada.

- Si el registro tiene relaciones, aparece un mensaje claro.
- Si el registro no tiene relaciones, se elimina correctamente.
- No aparece Whitelabel Error Page.
- No aparece error 500.

API REST

Situación	Código HTTP	Significado
Eliminación correcta	204 No Content	El registro se ha eliminado.
Registro inexistente	404 Not Found	El ID no existe.
Registro con dependencias	409 Conflict	No se puede eliminar por relaciones existentes.

Pruebas validadas

Conflictos controlados: DELETE /api/clientes/1, DELETE /api/empleados/1 y DELETE /api/proyectos/2 devolvieron 409 Conflict.

Eliminaciones correctas: DELETE /api/tareas/1, DELETE /api/empleados/1, DELETE /api/proyectos/1 y DELETE /api/clientes/1 devolvieron 204 No Content cuando ya no existían dependencias.

Registro inexistente: un segundo DELETE /api/tareas/1 devolvió 404 Not Found.

Conclusión

Cierre final de la memoria y relación del proyecto con el resultado de aprendizaje trabajado.

Con este proyecto se aprende que verificar una aplicación no consiste solo en mirar si arranca, sino en comprobar de forma ordenada que cada parte funciona como debe. A través de los casos de prueba, las pruebas unitarias, la prueba de integración y la depuración, se puede justificar mejor que el sistema responde correctamente.

Las pruebas ayudan a detectar errores antes de la entrega y permiten trabajar con más seguridad. También es importante separar las pruebas unitarias de las de integración, porque cada una revisa una parte distinta del proyecto.

La depuración con IntelliJ también ayuda mucho a entender el flujo interno del programa, porque permite ver paso a paso qué hace cada método y cómo cambian los datos. Además, la verificación final ha permitido detectar y corregir un error real de serialización en la API REST.

Por todo esto, SmartFenix cumple mejor el resultado de aprendizaje **RA3**, ya que se han comprobado casos normales y casos de error, la aplicación gestiona correctamente restricciones de base de datos y la eliminación de datos se realiza de forma segura. El sistema queda listo para entrega y demostración.